

Plan des Sprints

Plateforme intelligente de gestion des vulnérabilités — inspirée de Rapid7

Ce document présente, de manière condensée, les deux premiers sprints du projet : la mise en place de l'architecture technique (Sprint 0) et le développement du module d'authentification, de gestion des utilisateurs et du tableau de bord (Sprint 1).

SPRINT 0 — Préparation du projet

■ **Durée : 2 semaines**

■ **Objectif : Mettre en place l'environnement de développement, l'architecture technique et les outils DevOps, et produire les premiers livrables.**

Product Goal : disposer d'une architecture prête pour le développement des fonctionnalités métier.

User Stories

US0.1 — Création du projet

En tant que développeur, afin de démarrer le développement.

Critères d'acceptation :

- Projet Angular créé
- Projet Spring Boot créé
- Projet FastAPI créé
- Dépôts Git initialisés

US0.2 — Définition de l'architecture microservices

En tant que architecte, afin de garantir une architecture évolutive.

Critères d'acceptation :

- API Gateway
- Auth Service
- Asset Service
- Scan Service
- Vulnerability Service
- Alert Service
- Report Service
- AI Service

US0.3 — Mise en place de la base de données

En tant que développeur, afin de stocker les données du projet.

Critères d'acceptation :

- Base PostgreSQL créée
- Tables initiales : users, roles, assets, scans, vulnerabilities

US0.4 — Conteneurisation Docker

En tant que développeur, afin de standardiser les environnements.

Critères d'acceptation :

- Conteneurs créés : Angular, Spring Boot, PostgreSQL, Elasticsearch, Redis, FastAPI

US0.5 — Organisation Git

En tant que équipe technique, afin de structurer le travail collaboratif.

Critères d'acceptation :

- Branches définies : main, develop, feature/*, release/*, hotfix/*

US0.6 — Mise en place du CI/CD

En tant que équipe DevOps, afin de automatiser build et tests.

Critères d'acceptation :

- GitHub Actions configuré
- Build Angular
- Build Spring Boot
- Tests automatiques
- Build Docker

US0.7 — Rédaction de la documentation

En tant que équipe projet, afin de faciliter la prise en main du projet.

Critères d'acceptation :

- README
- Documentation d'architecture
- Guide d'installation
- Convention de code

Tâches techniques

- Initialisation Git
- Création du backend
- Création du frontend
- Création de l'API IA
- Docker Compose
- PostgreSQL
- Elasticsearch
- Redis
- Swagger
- Lombok
- Spring Security
- Angular Material

Livrables

✓ Architecture	✓ Elasticsearch
✓ Dépôts Git	✓ Documentation
✓ Docker	✓ CI/CD
✓ PostgreSQL	✓ Projet compilable

Définition de terminé (DoD)

- Projet compilable
- Tests de démarrage validés
- Documentation initiale disponible
- Déploiement local fonctionnel

SPRINT 1 — Authentification & Dashboard

■ Durée : 2 semaines

■ Objectif : Développer le module d'authentification, la gestion des utilisateurs et le tableau de bord de base.

User Stories

US1.1 — Authentification

En tant que utilisateur, afin de accéder à la plateforme.

Critères d'acceptation :

- Login / Logout
- Authentification par JWT + Refresh Token
- Mot de passe chiffré (BCrypt)
- Connexion réussie avec JWT généré
- Expiration du token gérée

US1.2 — Gestion des utilisateurs

En tant que administrateur, afin de gérer les comptes de la plateforme.

Critères d'acceptation :

- Rôles gérés : Administrateur, Analyste SOC, Responsable SSI, Auditeur
- Fonctions : ajouter, modifier, supprimer, désactiver un utilisateur

US1.3 — Gestion des rôles (RBAC)

En tant que administrateur, afin de contrôler les accès selon les responsabilités.

Critères d'acceptation :

- Permissions : lecture, écriture, administration

US1.4 — Tableau de bord

En tant que utilisateur, afin de visualiser rapidement l'état de la plateforme.

Critères d'acceptation :

- Indicateurs : nombre d'utilisateurs, d'actifs, de scans, de vulnérabilités, alertes critiques
- Graphiques : histogramme des vulnérabilités, répartition des actifs, statistiques générales

US1.5 — Journalisation

En tant que responsable SSI, afin de assurer la traçabilité des actions.

Critères d'acceptation :

- Historique des connexions / déconnexions
- Historique de création et suppression d'utilisateurs

US1.6 — API REST

En tant que développeur, afin de exposer les fonctionnalités backend.

Critères d'acceptation :

- Endpoints : /auth/login, /auth/logout, /users, /users/{id}, /roles, /dashboard

US1.7 — Interface Angular

En tant que utilisateur, afin de utiliser la plateforme via une interface web.

Critères d'acceptation :

- Pages : Login, Tableau de bord, Gestion des utilisateurs, Gestion des rôles, Profil utilisateur

Base de données

- users
- roles
- permissions
- user_roles
- audit_logs

Tâches par domaine

Backend	Frontend	DevOps
Spring Security	Layout, Sidebar, Navbar	Déploiement Docker
JWT / BCrypt	Login / Dashboard	Variables d'environnement
Validation & Swagger	CRUD Utilisateurs & Rôles	Sauvegarde PostgreSQL
API REST	Guards & Interceptors HTTP	Images Docker versionnées
Exceptions globales		

Tests

- Tests unitaires
- Tests API
- Tests Angular
- Tests JWT
- Tests CRUD

Livrables

✓ Authentification sécurisée	✓ API REST documentée
✓ Dashboard	✓ Interface Angular
✓ Gestion des utilisateurs	✓ Déploiement Docker
✓ Gestion des rôles	

Définition de terminé (DoD)

- Toutes les User Stories validées
- Tests unitaires réussis
- Couverture de tests minimale de 80 %
- Documentation API Swagger disponible
- Déploiement Docker fonctionnel

- Revue de code effectuée
- Démonstration au Product Owner validée